UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ing. en Tecnologías de la Información e Innovación Digital

Búsqueda con heurísticas: A* y Dijkstra

Tema: Ruteo óptimo en una red carretera del centro de México

Asignatura: Fundamentos de Inteligencia Artificial

Docente: Mtro. Javier García Lara

Grupo: 171

Equipo: Karina Paola, Denisse Valeria , Ronaldo Malcom

Fecha de entrega: 25 de octubre de 2026

1. Planteamiento del problema

Una empresa de transporte de carga necesita calcular la ruta de menor distancia entre ciudades del centro de México. Dada una red de 19 ciudades y 30 tramos carreteros, se busca el camino de costo mínimo entre un origen y un destino. Se comparan dos algoritmos: Dijkstra (búsqueda de costo uniforme, sin información del destino) y A* (búsqueda informada con heurística de distancia en línea recta).

Objetivos: (a) implementar ambos algoritmos en Python; (b) comparar costo, nodos expandidos y tiempo; (c) verificar la admisibilidad y consistencia de la heurística; (d) analizar qué ocurre si la heurística deja de ser admisible.

2. Formulación como problema de búsqueda

Estados: la ciudad en que se encuentra el vehículo. **Estado inicial / meta:** ciudad de origen / ciudad de destino. **Acciones:** viajar a una ciudad vecina conectada por carretera. **Costo de paso** c(n, n'): kilómetros del tramo. **Costo de camino** g(n): suma de km recorridos. El grafo es no dirigido y ponderado.

Heurística: h(n) = distancia haversine (línea recta sobre la esfera terrestre) entre n y la meta, calculada con las coordenadas de cada ciudad. Como ningún camino por carretera puede ser más corto que la línea recta, h(n) ≤ h*(n): es admisible. Además cumple la desigualdad triangular, por lo que es consistente: h(n) ≤ c(n, n') + h(n').

Datos: coordenadas aproximadas del centro de cada ciudad; distancias por carretera aproximadas y redondeadas (archivo red_carretera.py). Se evaluaron 6 consultas.

3. Procedimiento

1) Se construyó la lista de adyacencia. 2) Se implementaron Dijkstra y A *con cola de prioridad (heapq) y lista de cerrados; Dijkstra ordena por g(n), A* por f(n) = g(n) + h(n). 3) Para cada consulta se registró costo, ruta, nodos expandidos y tiempo medio sobre 2000 ejecuciones. 4) Se verificó h(n) ≤ h *(n) calculando h*(n) con Dijkstra desde la meta, y la consistencia arista por arista. 5) Se repitió A* con f = g + w·h (w = 1.25, 1.5, 2, 3) para observar el efecto de perder admisibilidad.

Núcleo de A* (código completo en el repositorio):

```
def a_estrella(grafo, inicio, meta, h=haversine, peso=1.0):
    g = {inicio: 0.0}; padre = {inicio: None}
    frontera = [(peso * h(inicio, meta), 0.0, inicio)]
    cerrados, orden = set(), []
    while frontera:
        _, costo, n = heapq.heappop(frontera)
        if n in cerrados: continue
        cerrados.add(n); orden.append(n)
        if n == meta:
            return _reconstruir(padre, meta), g[n], len(orden), orden, ...
        for vecino, w in grafo[n]:
            nuevo = g[n] + w
            if nuevo < g.get(vecino, math.inf):
                g[vecino] = nuevo; padre[vecino] = n
                heapq.heappush(frontera, (nuevo + peso*h(vecino, meta), nuevo, vecino))
```

4. Resultados

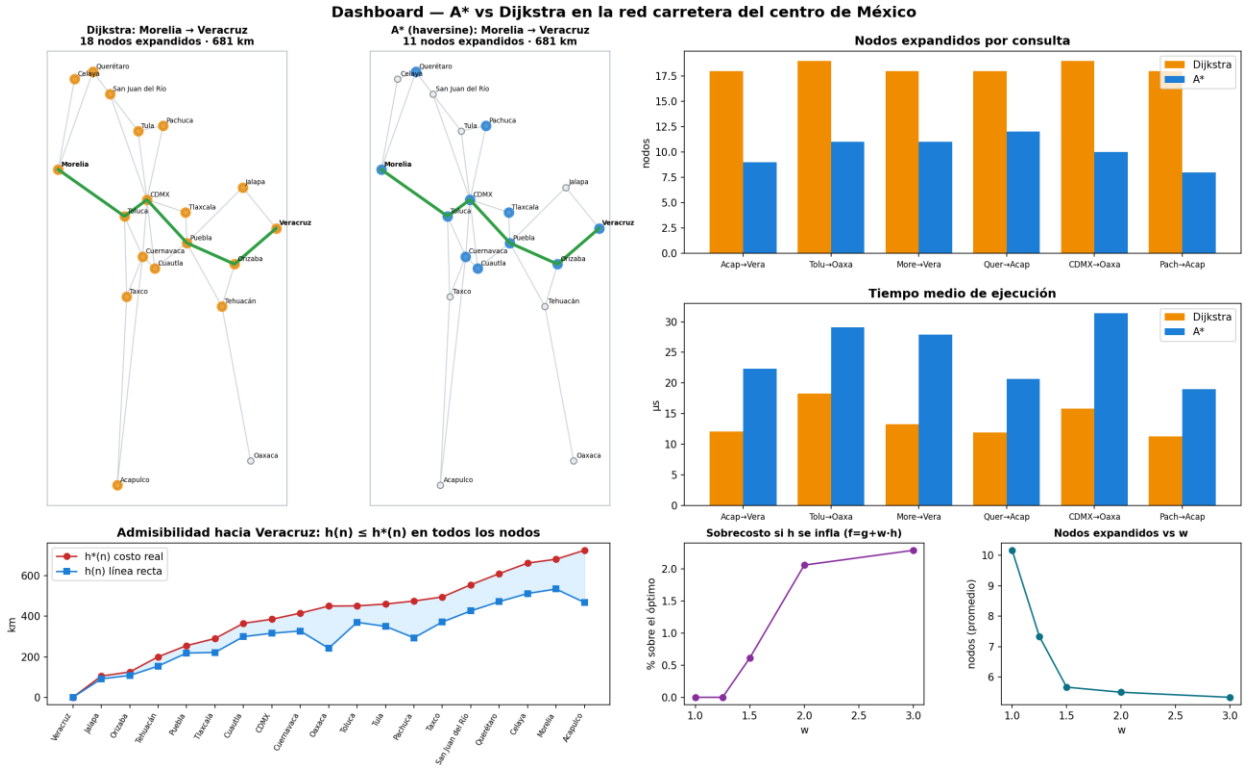
Tabla 1. Comparación A* vs Dijkstra (ambos hallan el mismo costo óptimo).

Origen → Destino	Costo (km)	Nodos Dijkstra	Nodos A*	Reducción	t Dijkstra (µs)	t A* (µs)
Acapulco → Veracruz	725	18	9	50.0%	12.1	22.3
Toluca → Oaxaca	566	19	11	42.1%	18.3	29.1
Morelia → Veracruz	681	18	11	38.9%	13.3	27.9
Querétaro → Acapulco	620	18	12	33.3%	11.9	20.7
CDMX → Oaxaca	500	19	10	47.4%	15.8	31.4
Pachuca → Acapulco	485	18	8	55.6%	11.3	19.0

Tabla 2. Rutas óptimas encontradas.

Consulta	Ruta
Acapulco → Veracruz	Acapulco → Taxco → Cuernavaca → Cuautla → Puebla → Orizaba → Veracruz
Toluca → Oaxaca	Toluca → CDMX → Puebla → Tehuacán → Oaxaca
Morelia → Veracruz	Morelia → Toluca → CDMX → Puebla → Orizaba → Veracruz
Querétaro → Acapulco	Querétaro → San Juan del Río → CDMX → Cuernavaca → Taxco → Acapulco
CDMX → Oaxaca	CDMX → Puebla → Tehuacán → Oaxaca
Pachuca → Acapulco	Pachuca → CDMX → Cuernavaca → Taxco → Acapulco

Figura 1. Dashboard comparativo (mapas de exploración, nodos, tiempo, admisibilidad y heurística inflada).



5. Análisis

Eficiencia en nodos

A* expandió en promedio 44.6% menos nodos que Dijkstra (entre 33.3% y 55.6%) y siempre devolvió el mismo costo. Dijkstra explora en todas direcciones porque no sabe dónde está la meta; la heurística de línea recta orienta la búsqueda y evita, por ejemplo, visitar Celaya o Querétaro cuando se viaja de Morelia a Veracruz.

Tiempo de ejecución

Con estos datos A *fue más lento en tiempo real (≈ 19–31 µs vs 11–18 µs). No es una contradicción: el grafo tiene sólo 19 nodos, así que ahorrar unas cuantas expansiones no compensa el costo de calcular senos y cosenos del haversine en cada evaluación de h(n). La ventaja de A* se vuelve notoria en grafos grandes (miles de nodos, por ejemplo una red vial real), donde el número de expansiones domina el tiempo. Ésta es una limitación del experimento y debe tomarse en cuenta al interpretar la Figura 1.

Admisibilidad y consistencia

Para los tres destinos usados (Acapulco, Oaxaca y Veracruz) se encontraron 0 violaciones de h(n) ≤ h*(n) y 0 violaciones de consistencia. Por eso A** garantiza el óptimo y no necesita reabrir nodos cerrados. Esta propiedad depende de que cada tramo en la tabla de datos sea mayor o igual que la distancia en línea recta entre sus ciudades; si se capturara un tramo con km menores a la línea recta, la heurística dejaría de ser admisible.

¿Y si la heurística no es admisible?

Al multiplicar h por w (f = g + w·h) los nodos expandidos bajan de 10.2 (w = 1) a 5.7 (w = 1.5) y 5.5 (w = 2), pero aparece sobrecosto: 0.61% con w = 1.5 y 2.06% con w = 2 sobre el óptimo. Es el compromiso clásico velocidad vs optimalidad: con w > 1 se pierde la garantía de la ruta mínima a cambio de explorar menos. En esta red el sobrecosto es pequeño porque la red es casi lineal y tiene pocas rutas alternativas.

Complejidad

Con montículo binario ambos algoritmos son O((V + E) log V) en el peor caso; A* con heurística consistente nunca expande más nodos que Dijkstra y, en la práctica, expande bastantes menos. La memoria es O(V) en ambos casos.

6. Conclusiones

1) A *y Dijkstra encontraron rutas de igual costo en las 6 consultas; A** redujo la exploración en 44.6% en promedio. 2) La heurística haversine es admisible y consistente en esta red, lo que garantiza optimalidad. 3) En un grafo tan pequeño el costo de evaluar h(n) hace a A* más lento en microsegundos; su beneficio real se espera en redes grandes. 4) Inflar la heurística reduce nodos pero sacrifica optimalidad. 5) Como trabajo futuro: usar una red vial real (OpenStreetMap), tiempos de viaje en lugar de km y comparar con otras heurísticas.

Declaración de uso de IA

Herramienta: Claude (Anthropic). **Prompt:** «ayúdame a hacer la actividad: resolver un caso real de ruteo con A *y Dijkstra, con dashboard y análisis de admisibilidad». Uso: generación del borrador del código, de los datos de la red, del dashboard y de la estructura del reporte. Verificación:** [el equipo

debe describir aquí qué revisó: ejecución del código, revisión de distancias en un mapa, lectura del análisis].

Referencias (APA 7.ª ed.)

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107.

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.